

How much work can one AI agent safely carry? A measured answer — and what to build with it

- **What:** 460 episodes across two task worlds, mapping exactly where an LLM agent's internal picture of the world stops being good enough to act on.
- **Found:** a collapse boundary that is sharp, cheap to measure — and different in every environment. No universal formula exists; the measurement recipe is the asset.
- **Next:** a product wedge that needs none of the theory to sell: stop agents from writing to the wrong real-world object.

Repo `effective-state-load` Model `deepseek/deepseek-v4-flash` · temp 0.0 Total spend ~\$16 at list rates Date 2026-07-05

The problem this experiment attacks

- Agents don't fail by crashing. They fail by **acting on a wrong picture of the world** — confidently.
- The picture degrades as the task grows: more things to track, more conditions per action. Past some load, the agent burns its budget on invalid moves and delivers nothing.
- Nobody can see this coming today. Token dashboards and retry counters are **lagging** indicators; by the time they move, the money is spent.

- **The orchestrator's daily question:** give this task to one agent, or split it across three? Today that's answered by vibes and token caps.
- **The cost asymmetry that makes it matter:** a failed long-horizon episode costs 4–6× more than a successful one *and delivers nothing*. Collapse cost dominates token cost.
- **E.g.** an agent updating a customer record passes every schema check — with the right *type* of ID belonging to the wrong customer. That write is the failure mode this project measures, predicts, and (in the product section) blocks.

The two dials that define "load"

SC State cardinality — how much must be remembered

The number of live things — objects, records, facts, goals — the agent has to keep straight **at the same time**, updating each one as the world changes.

E.g. a coding agent mid-refactor tracks ~30 live items: files edited, functions renamed, tests failing, a half-applied migration. SC counts those items. Forget one, and it edits a file it already changed.

DD Dependency density — how much must be true at once

The number of conditions that must **all hold simultaneously** before an action is allowed to work.

E.g. `create_payment_link(invoice_id, tax_quote, approval_token, region)` works only if the invoice exists and the quote is current and the approval hasn't expired and the region matches. DD counts the ands. One stale input, and the call fails.

Effective State Load (ESL) is the hypothesis under test: that one number, $SC \times DD$, predicts when the agent's world model collapses — a dispatch score an orchestrator could budget against.

Three research questions, three phases

RQ1 · PHASE 1

Is collapse real and measurable?

- Does an agent's world model degrade under SC/DD load in a reproducible pattern?
- Does the world model fail **before** the actions do (the paper's core claim)?
- Method: 240 episodes of StatefulPuzzle over a 6×4 SC/DD grid.

RQ2 · PHASE 2

Does one number compress it?

- Do cells with equal $SC \times DD$ behave alike?
- Does it beat simpler predictors — SC alone, SC + DD, raw token count?
- Method: contour/bucket analysis + a weighted $SC^a \times DD^b$ calibration.

RQ3 · PHASE 3

Does it transfer to tool use?

- Same score, different world: typed tool-calling with provenance rules.
- Is the formula environment-general, or does every workload need its own calibration?
- Method: 160 episodes of ToolDAG-B behind a locked go/no-go gate.

460

episodes total

240 puzzle + 60 gate + 160
transfer

~10,000

steps logged

full believed-state vs gold-
state traces

~\$16

entire program, list rates

the boundary is cheap to
map

1

parse failure in 10k steps

the signal is not a
formatting artifact

RQ1 — Yes: collapse is real, sharp, and not an artifact

- The surface has three regimes: a **safe plateau** (perfect through $SC \leq 15$), a **bending transition** (DD starts costing once $SC \geq 20$), and a **collapse floor** ($SC = 40$).
- **Every failure is the agent's own.** A goal-directed oracle solves all 24 cells 10/10 on the same seeds — the tasks are provably solvable within the horizon.
- Zero parse failures in 5,434 steps. The paper's world-model-fails-first signature reproduces (87.5% of paired collapses, median 3 steps early) at this model's operating point.

StatefulPuzzle — final success (of 10)

	DD=1	DD=2	DD=4	DD=6
SC 5	10	10	10	10
SC 10	10	10	10	10
SC 15	10	10	10	9
SC 20	10	10	9	7
SC 25	10	10	7	4
SC 40	1	3	0	0

Diverging scale · gray ≈ the 50% boundary · blue safe / red collapsed

What rules the alternatives out

Task too hard for the horizon?	No — oracle 10/10 in every cell
JSON/formatting failure?	No — 0 parse failures / 5,434 steps
Output truncation?	No — 3 ceiling hits / 240 episodes
Random noise?	No — every $SC \leq 15$ cell 10/10; every $SC=40$ $DD \geq 4$ cell 0/10

Full evidence: [docs/reports/phase1-scdd-surface.md](#) and the Phase 1 slide deck.

The anomaly that changes what you should monitor

Agents fail while their state accuracy dashboard reads 99.5% green.

StatefulPuzzle SC=25 / DD=6: 6 of 10 episodes fail — with a 99.5%-accurate believed world state.

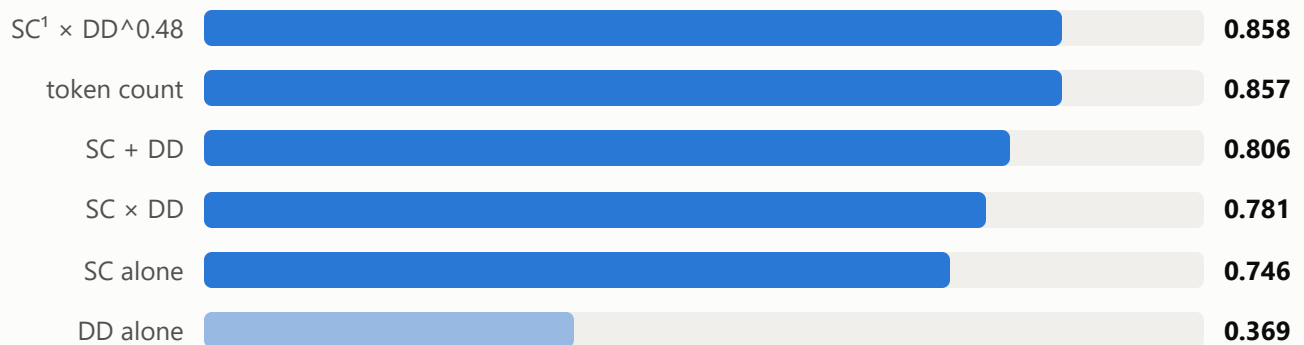
- **Why:** at ~31 live entities, a handful of wrong facts out of hundreds barely moves the accuracy average — but it's enough to break precondition chains. The agent wastes its step budget on invalid moves and repeated navigation to wrong rooms.
- **The signals that do move first:** binding errors, precondition violations, state staleness, retry inflation — per-action correctness, not aggregate accuracy.

- **Consequence for anyone running agents:** "state accuracy" style metrics are the wrong alarm. Collapse announces itself in the *leading* indicators — if you're instrumented to see them.

RQ2 — Partially: the score works, with calibrated weights

- **Simple $SC \times DD$ is not a clean law in this world.** Equal-ESL cells split by up to 90 points of success at the boundary; ($SC=40$, $DD=1$) at ESL 40 fails while ($SC=25$, $DD=2$) at ESL 50 is perfect. SC dominates here.
- **Calibration fixes the ranking:** the best fit is $SC^1 \times DD^{0.48}$ — state weighs about twice as much as coupling. Spearman vs collapse risk: 0.858.
- **The humbling result:** free-to-measure token count ties it (0.857). The formula's raw predictive edge is zero *in this environment* — a fact the product section takes seriously.

Predictor ranking — StatefulPuzzle (Spearman vs risk)



What the calibration settles

- DD is not noise: at $SC=25$, raising DD from 2 to 6 walks success from 100% to 40%. The boundary genuinely bends.
- But the weights are the finding — **$SC \approx 2 \times DD$ in this world** — and weights are exactly what a one-size formula can't carry across worlds.
- Token count ranks well but is measured *after* the run and says nothing about *how to split* a task. The formula's value must be actionability, not prediction. That claim gets tested by RQ3 — and priced by the market section.

RQ3 setup — a tool world where identity matters

- **Why build ToolDAG-B:** the upstream tool environment only checked argument *types* — any well-typed value passed. Production agent failures aren't type errors; they're **identity errors**: a

valid-looking ID from the wrong place.

- **The change:** every tool call must name its exact input variables, each input must come from the *designated producer* (its provenance), and same-type decoy variables are seeded to scale with SC. DD becomes the number of bindings that must be simultaneously right per call.
- **Disciplined spend:** a locked go/no-go gate ran first — full grid money only if dependency pressure measurably bites.

Gate verdict: PASS — a 100-point cliff

DD (AT SC=20)	1	2	3	4	5	6
success	10/10	10/10	1/10	0/10	0/10	0/10

- Monotone, max drop 100pp — far exceeding the locked ≥ 20 pp rule.
- Failures are the designed mode: wrong-provenance + binding errors, not formatting.

Why this world maps to production

Schemas machine-check types for free. What no schema can express: "*a customer_id of the right type but the wrong customer.*"

E.g. in manufacturing this is the classic bill-of-materials error — right part type, wrong approved part number or wrong revision. Provenance is the thing that broke.

RQ3 — The transfer result: same model, opposite cliffs

- Same model, same harness, same horizon, same SC/DD labels. **The dominant axis flipped.**
- StatefulPuzzle: a **state wall** — SC=40 collapses even at DD=1. ToolDAG-B: a **coupling cliff** that moves earlier as SC grows — and SC=40 sails through at DD=1.
- Every ToolDAG-B cell is 10/10 or 0/10. The boundary is one DD step wide.

StatefulPuzzle — SC-dominant

	DD=1	DD=2	DD=4	DD=6
SC 5	10	10	10	10
SC 10	10	10	10	10
SC 20	10	10	9	7
SC 40	1	3	0	0

Rows SC 15/25 omitted for like-for-like comparison · collapse arrives via the SC axis

ToolDAG-B — DD-dominant, SC×DD interaction

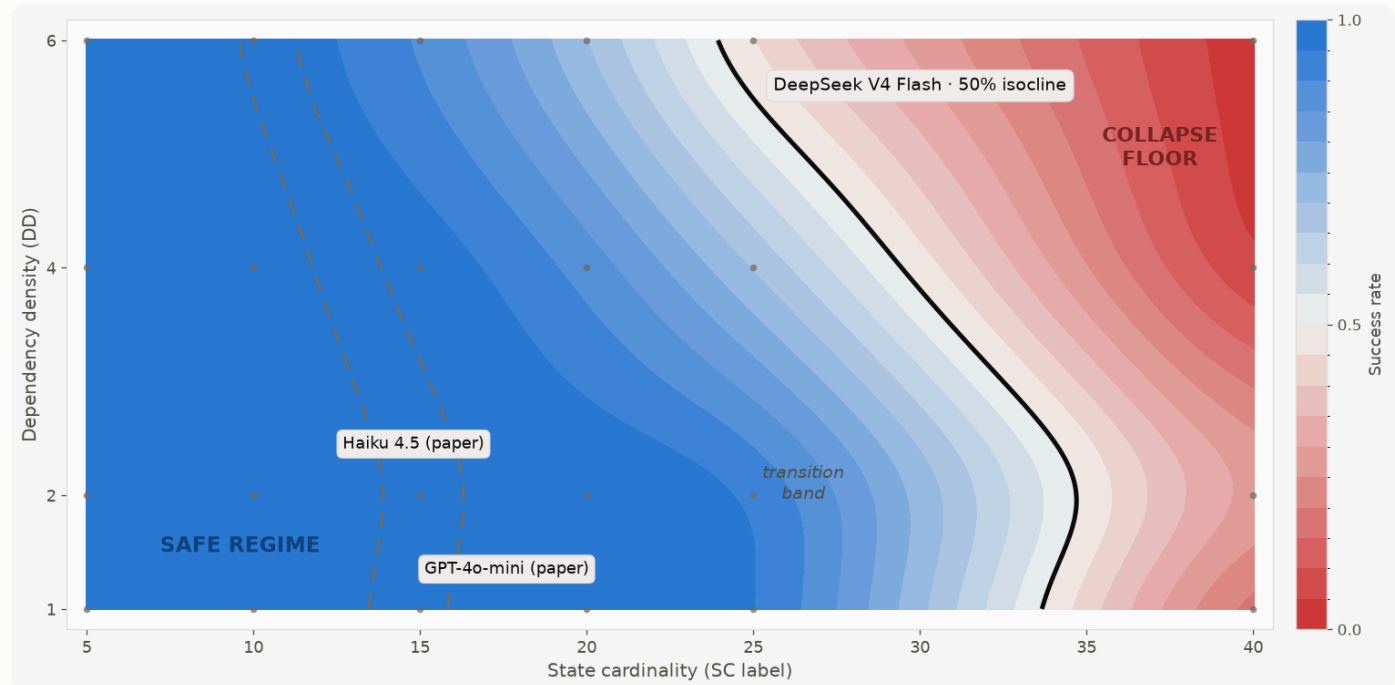
	DD=1	DD=2	DD=4	DD=6
SC 5	10	10	10	10
SC 10	10	10	10	10
SC 20	10	10	0	0
SC 40	10	0	0	0

160/160 unique episodes · every cell 10/10 or 0/10 · the cliff moves left as SC rises

The phase diagram — one model, two collapse boundaries

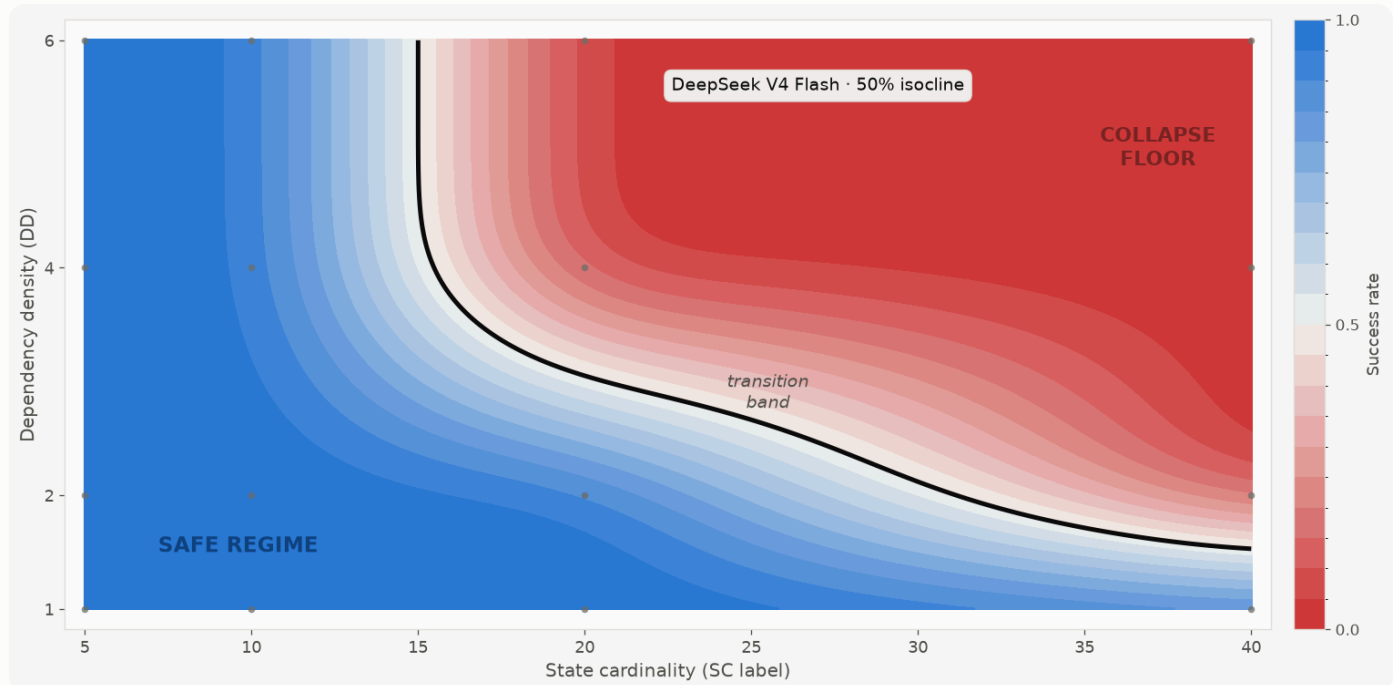
- The bold curve is the **50% isocline** — the collapse boundary at this model's operating point.
- StatefulPuzzle's boundary is a **state wall**: nearly vertical near $SC \approx 34$, tilted only slightly by coupling.
- ToolDAG-B's boundary is an **L-shaped coupling cliff**: safe at any measured state while $DD=1$, then the wall races in to $SC \approx 15$ once $DD \geq 4$.

StatefulPuzzle — the state wall



Success rate interpolated and smoothed from the 24 measured cells (dots, $n=10$ each). Dashed curves: where the paper's models collapsed — drawn with the measured boundary's shape at the paper's fig. 1 anchor positions (illustrative; the paper publishes no isocline coordinates). Capability moves the cliff; it doesn't remove it.

ToolDAG-B — the coupling cliff



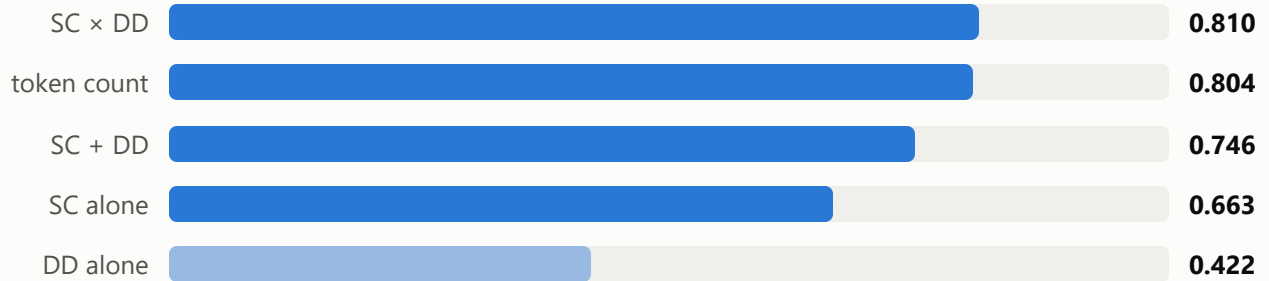
Interpolated from the 16 measured cells (dots, $n=10$ each); every measured cell is 10/10 or 0/10. Safe at every measured SC while $DD=1$ — including $SC=40$, the cell class that died in StatefulPuzzle. Generated by

`analysis/openrouter/phase_diagram.py`.

The verdict: a family, not a formula

- **Inside ToolDAG-B, simple $SC \times DD$ is the best predictor** (Spearman 0.810 — beating token count, 0.804) and separates the surface *perfectly*: every cell at $ESL \leq 60$ passes, every cell at $ESL \geq 80$ fails.
- **But that same threshold is nonsense in StatefulPuzzle**, where $ESL\ 40$ already collapses ($SC=40, DD=1$) and $ESL\ 50$ is perfect ($SC=25, DD=2$).
- **So: environment-general as a metric family and measurement recipe — not as a portable number.** The boundary's location and the axes' weights must be calibrated per workload. One cheap grid (\$4–7) does it.

Predictor ranking — ToolDAG-B (Spearman vs risk)



- Equal-ESL cells behave identically here (bucket variance 0.0000).
- The multiplicative form wins in the world built around simultaneous bindings — evidence the family is right even though the weights move.

Why "no universal formula" is the useful answer

- If one formula worked everywhere, it would be a library function — free within a year, no business.
- Because weights are workload-specific, the durable asset is the **repeatable measurement**: grid → surface → calibrated frontier → operating margin.
- The analogy: no universal server-capacity number exists either — and load testing is a durable category anyway.

Anatomy of a collapse — what actually breaks

- **The failure is identity binding, overwhelmingly.** Across all 2,811 transfer-grid steps: 1,328 variable-binding errors + 1,327 wrong-provenance errors — versus **one** parse error.
- **Memory and correctness fail on different axes.** Believed-state accuracy falls with SC (0.55 at SC=40) even in cells that succeed 10/10 — while success is governed by DD. The agent can hold a degraded map and still win, as long as few things must be right *simultaneously*.
- **The world model fails before the actions do** — here at the pre-registered threshold, no sensitivity analysis needed: 43 paired collapses, 67% world-state-first, median 1 step early.

Plain-language version: under load, the agent doesn't forget everything — it quietly holds a *plausible but wrong* graph of which value came from where. Every schema check passes. Then one action needs three of those facts to be right at once, and the chain breaks. That's why the fix that works isn't "bigger memory" — it's checking provenance *outside* the model. A perfectly informed model still failed these cells with the check available to it in-context.

What it means #1 — cliff economics, not a gradual tax

- **Margin is nearly free.** Inside the boundary, cost per delivered task is flat: \$0.004–0.008 at low state, \$0.015–0.025 at high-but-safe load. Backing two DD steps away from the cliff costs almost nothing.
- **Overload is worse than infinite retries.** Past the cliff, episodes cost 4–6× more per attempt (failures burn the whole horizon) and deliver zero. Retrying doesn't help — success is 0/10, not 3/10.
- **The boundary is a band, not a line.** Identical temp-0 reruns flipped 6 of 16 boundary-cell outcomes. Operate a margin *inside* the band — the fractional-Kelly rule for agents.

\$0.005

per delivered task, safe regime

ToolDAG-B $SC \leq 10$, any DD · flat

1.4–2.5×

retry inflation at the boundary band

StatefulPuzzle transition cells

4–6×

cost per attempt past the cliff

\$0.08–0.10/episode, full horizon burned

∞

cost per delivered task past the cliff

0/10 success — nothing to amortize

There is also a **predictability dividend**: outcome variance is $p(1-p)$ — near zero deep in the safe regime, maximal at the boundary. Chains of subtasks multiply success rates, so one boundary-band stage poisons a whole pipeline's reliability. Safe-regime staging is what makes agents SLA-able.

What it means #2 — design rules for agent systems

PACKING RULE

Split state, keep coupling

- The frontier is asymmetric: state (SC) is the axis to distribute across agents; coupled conditions (DD) belong *inside* one slice.
- E.g. don't give two agents halves of one tightly-coupled change; give each a subsystem whose dependencies stay internal.

BUDGET RULE

The load doesn't vanish — it moves up

- Dependencies split across agents reappear as the *orchestrator's* SC/DD.
- Load budgeting must apply recursively, at every level of the tree — including the top.

MARGIN RULE

Stay inside the band

- Cliff + band + cheap margin $\Rightarrow$ never operate at measured capacity.
- Better models move the cliff outward; they don't remove it. Recalibrate on every model/prompt/tooling change.

MONITORING RULE

Watch leading indicators

- Aggregate state-accuracy stays green through collapse.
- Instrument binding errors, precondition violations, staleness, retry inflation — per-action, not per-average.

Can you actually measure SC/DD outside a lab? Three scenarios

- The units are arbitrary — what matters is counting the **same way twice**: once when calibrating the frontier, once when sizing a real task against it.
- And only **ordinal accuracy with margin** is needed: the cliff is one step wide, so the question is "safe side, with room?" — not "is SC exactly 23?"

SCENARIO	WHERE IT APPLIES	INGREDIENTS NEEDED	HOW YOU GET SC / DD
Count	State already lives in a system of record — manufacturing, finance ops, CRM writes	The database, a defined task scope, its consistency rules	A query. E.g. an engineering change touches N records (SC); each update must hold revision + vendor + effectivity consistent (DD)
Measure	Agent works through tools you control — coding, IT automation	A middleware interception point, a live provenance graph, stable IDs in tool outputs	Observed at runtime: live objects in the graph (SC); required-correct bindings per call (DD). React mid-run as load climbs
Estimate	Task arrives as plain language, pre-dispatch	A plan graph, or a cheap LLM estimator, plus outcome history to calibrate it	Count over the plan (entities $\approx$ SC, max preconditions per step $\approx$ DD); correct the estimator against measured outcomes over time

The adoption path is the same list in reverse: start wherever the customer's ingredients allow, and funnel toward instrumented measurement — which is also the guardrail product surface (next slide).

Productization — five shapes, one wedge

The shapes, ranked by evidence in hand

1 • Runtime provenance guardrails	validate every agent write against where its data came from — the failure mode with the 100-point cliff behind it
2 • Collapse-aware observability	leading indicators (bindings, staleness, retries) instead of green-through-collapse accuracy dashboards
3 • Capacity calibration as CI	re-map a customer's collapse boundary on every model/prompt change — recalibration is recurring by nature
4 • Dispatch-time load budgeting	the orchestrator vision — gated on proving ESL routing beats token-cap routing (experiment designed, unrun)
5 • Provenance eval suite	"which cheap model holds binding at DD=4?" — marketing collateral more than revenue

THE WEDGE • NEEDS NO BELIEF IN ESL

Stop agents writing to the wrong real-world object

- **Positioning:** "Schema validation checks that it's a customer ID. **We check that it's the right customer.**"
- **The real enemy** isn't schema tools — it's the human review queue. The pitch that closes: *verify every write against its provenance, so you can un-gate more automation.*
- **Who buys:** platform teams running agents that write to CRM / ERP / billing / ticketing, currently gating those writes with human review.
- **Lifecycle = trust modes:** Shadow (audit only) → Gate (flag or block one risky tool) → Un-gate (evidence-backed removal of review).

The strategic link: the guardrail's provenance graph is the SC/DD measurement instrument. Load metering, collapse dashboards, and dispatch budgeting become upsells on data the guardrail already collects. Sell the smoke detector first; the fire-risk model comes with the subscription.

MVP / proof-of-concept path — and the honest caveats

The path, cheapest-first

- 1 **Shadow audit prototype.** Replay real traces through a provenance checker; deliver "your agents almost made N wrong-object writes last month." **Time-to-value target: first verified near-miss within one hour, zero production changes.** This simultaneously tests the market's weakest assumption — that these failures happen in the wild at all.
- 2 **Entity-mapping UX on three real, ugly tool catalogs.** The only bespoke onboarding step; if it can't stay under ~30 minutes, the product crosses into consulting.
- 3 **The routing experiment (~\$10–20).** ESL-based task routing vs token-cap routing on completion-per-dollar, second model, realistic workload. This is the gate for everything beyond the guardrail.
- 4 **Second-model gate sweep (~\$1).** Buys the cross-model shape criterion the budget deferred.

Caveats & kill criteria — written down in advance

- **Evidence limits:** one model, synthetic environments, 10 seeds per cell. The share of *real-world* agent incidents caused by state load is unmeasured — the single weakest link.
- **Kill #1:** shadow audits at 3 real customers find ~zero provenance inconsistencies → the failure mode is synthetic; stop.
- **Kill #2:** token-cap routing matches ESL routing on outcomes → guardrail only; never build the budgeting SDK.
- **Kill #3:** a frontier-model generation moves the cliff $>5\times$ → boundary economics die; the guardrail survives on its own merits.

Generated 2026-07-05 · effective-state-load, Phases 1–3 complete · 460 episodes · deepseek/deepseek-v4-flash via OpenRouter, temp 0.0, T=40 · companion docs: [docs/reports/phase1-scdd-surface.md](#) · [docs/reports/phase3-transfer.md](#) · [docs/product-notes.md](#) · Phase 1 deck: [phase1-report.html](#) · paper: [World-Model Collapse as a Phase Transition](#)

Glossary

SC — state cardinality	how many live things (objects, records, facts, goals) the agent must track simultaneously
DD — dependency density	how many conditions must all hold at once for a single action to be valid
ESL — effective state load	the metric family under test: $SC \times DD$ (or calibrated $SC^a \times DD^b$) as a task-difficulty score
World model / believed state	the agent's own running picture of the world, updated each step from what it observes
Gold state	the environment's ground-truth state, tracked deterministically for comparison
WSA — world-state accuracy	per-step agreement between believed state and gold state
Collapse	the regime where a degraded world model makes the agent burn its budget on invalid actions and deliver nothing
Isocline / collapse boundary	the curve through $SC \times DD$ space where success crosses 50%
Safe regime / band	load comfortably inside the boundary; the band is the boundary's width — identical reruns flip outcomes inside it
Provenance	which producer a value came from — the initial pool or a specific earlier tool call

Binding error	using a variable of the right type but the wrong identity or origin — ToolDAG-B's designed failure mode
tau_w / tau_a / lag	first step believed-state accuracy drops below threshold / first invalid action / their difference — negative lag means the world model failed first
Episode · cell · seed	one full task run · one (SC, DD) grid point (10 episodes each) · the deterministic task-generation key
Horizon (T)	the step budget per episode — 40 throughout, oracle-verified as sufficient
Oracle ceiling	success rate of a scripted perfect-information solver on the same tasks — separates agent failure from unsolvable tasks
Label vs realized load	the SC/DD knob settings vs what the environment actually produced — analyses run on both
Spearman correlation	rank agreement between a predictor and observed collapse risk (1.0 = perfect ordering)
JSON fallback rate	share of model calls where structured output needed a fallback parse — the formatting-corruption check
Fractional-Kelly margin	operating deliberately inside the measured boundary because margin is cheap and overload is a cliff
Orchestrator	the system that decides how to split work across agents — the intended consumer of a load budget

References & artifacts

TYPE	REFERENCE
Paper	<i>World-Model Collapse as a Phase Transition</i> . arXiv:2606.31399. arxiv.org/abs/2606.31399 — source of the SC/DD stress framework, the environments, and the world-state-first hypothesis.
Reference code	Hik289/world-model-collapse , vendored byte-identical at commit <code>66c12c3</code> (MIT). Provenance record: <code>docs/UPSTREAM.md</code> .
Model	<code>deepseek/deepseek-v4-flash</code> via OpenRouter · \$0.09/M input, \$0.18/M output (list, verified 2026-07-03) · temperature 0.0, decoding seed 42.
Design docs	<code>docs/plan.md</code> (experiment design, §8 decision rule, §12 ToolDAG-B spec + locked gate, §15 report template) · <code>CONTEXT.md</code> (vocabulary, standing decisions) · <code>docs/guide.md</code> (runbook).
Evidence reports	<code>docs/reports/phase1-scdd-surface.md</code> (Phase 1 surface + oracle audit) · <code>docs/reports/phase3-transfer.md</code> (transfer comparison + verdict) · <code>docs/reports/phase1-report.html</code> (Phase 1 deck).
Data	<code>data/raw_logs/*.jsonl</code> (step/episode logs + cost trackers, all runs) · <code>data/dedup_*.jsonl</code> (Phase 1 deduplicated) · <code>outputs/openrouter/<run>/</code> (analyzer summaries, heatmap CSVs/PNGs, traces, ledgers).
Analysis code	<code>analysis/openrouter/</code> : <code>analyze_grid.py</code> · <code>calibrate_esl.py</code> · <code>check_gate.py</code> · <code>dedup_logs.py</code> · <code>operating_point.py</code> · <code>phase_diagram.py</code> . Configs: <code>configs/*.json</code> .
Decision record	Issues #6 (Phase 1), #7 (Phase 2 verdict gate), #8 (Phase 3 + ToolDAG-B design review) · PRs #16–#20.
Product record	<code>docs/product-notes.md</code> — evidence-ranked opportunity shapes, market map, adversarial pass, wedge, kill criteria.